$_FILES['userfile']['size'] to check the file size, *$_FILES['userfile']['type']* isn't reliable to know the type, since it's browser-provided. Use *mime_content_type()* for that.

Redirect URLs

Sometimes when we send visitors to a page to go through some process, we might want them to return or go to another page after completing that task. For example, we might want someone to be redirected to */dashboard.php* after logging in. Since the destination may vary, this target location cannot be put in the code. To solve this, we pass the target URL to *login.php* as follows:

login.php?redirect=/dashboard.php.

Or we can use cookies for the same purpose.

Finally, the redirection is accomplished by inserting code like what follows in the on-success part of *login.php:*

```
if(isset ($_GET['redirect']))
    header ('Redirect: ' . $_
GET['redirect']);
else
    header ('Redirect: /');
```

This will work fine. The problem is that an attacker can create a login page URL like the following and send it to innocent users to visit: *https://yoursite.com/login.php?redirect=https://attcakersite.com/success.php*.

Visitors will be concerned about the login page only. If it looks legit, they won't try to validate the subsequent pages.

The solution is to validate the redirect URLs before setting the header, or to accept code names (e.g., *dashb*) instead of full URLs and map them to legit URLs before including them in the header.

SQL injection

Don't worry; I'm not going to elaborate on this topic. For those who haven't heard about it yet, SQL injection is a process by which a user enters cleverly formatted text that gets executed as SQL when embedded in your query statements. To prevent this, whenever you earlier used the following code:

```
query( '... WHERE something="'.
    $_POST['something'].'"' );
```

…now use the code below, instead:

```
query( '... WHERE something='.
    mysqli_escape_string($_
POST['something']) );
```

Or even better, use something like PDO prepared statements.

Configuring PHP

Before going further, let us check how to tweak the PHP settings. PHP configurations are stored in the file *php.ini* and in related ones. Write a simple script that calls *phpinfo()* and visit it from the browser to know their locations.

The main file may not be accessible in shared hosting environments, but many settings can be set in the beginning part of your scripts using *ini_set()*. You can see a usage example in the next section.

Session hijacking

We all know what a session is. HTTP is stateless and a session helps us to remain logged in (or identified as the same person) while hopping from one page to another in the same site. Once logged in, the server sets a session-identifying cookie in our browser, and the browser sends this back each time we visit a page in the same site.

This means that if an attacker manages to steal the cookie of a legit logged in user, he can place it in his browser and log in as the same user. Session hijacking can also be done by other means, but let's start with cookie stealing.

One way to steal a session cookie is to perform XSS. Attackers first create a profile in the site, inject some script to their own profile, and when others visit their profile, the code gets executed in the browser of the visitors. The attacker can use *document.cookie()* to get the victim's session cookie, and get it sent to him using Ajax.

There are various measures to prevent session hijacking techniques that include cookie stealing:

- Use HTTPS
- Tell the browser to safeguard the cookie
- Make sure the server-side session data is inaccessible to other sites
- Use session IDs that cannot be guessed
- Regenerate the session ID periodically

The first one is a must. We'll discuss the third precaution later. PHP session IDs aren't that easy to guess, so the fourth preventive measure is also checked. Regarding the fifth one, there is a function called *session_regenerate_id()* to regenerate the session ID, but it can harm the user experience.

What about the second precaution, then? Normally, you shouldn't trust the browser since the user can modify it or can create a custom one. Plus, there can be malware. However, since cookie stealing is never done by somebody against oneself and we're trying to protect the user from third-party scripts, we can trust the client-side in this case (unless there is malware).

The following *php.ini* settings will be a good start:

```
; Prevent access from JavaScript
; (no longer accessible via document.
cookie)
session.cookie_httponly=On

; Delete the cookie when the browser is
closed.
session.cookie_lifetime=0

; Do not accept the session ID from
GET/POST/URL
session.use_cookies=On
session.use_only_cookies=On

; Do not accept a session ID provided
```